



《人工智能基础》课程实验报告

学院系别	信息学院
专业名称	人工智能
学生姓名	孟媛
学生学号	22920202200773
任课教师	苏畅

一、A*算法求解野人传教士问题

（一） 问题描述

河岸的一侧有野人与传教士各 m 个，有一条能容纳 n 个人的船。传教士与野人都会划船，现在要求任何情况下，野人数不能超过传教士数。设计一个程序，对于给定的 m, n 输出最少的划船次数。

（二） 算法介绍

A* (A-Star)算法是一种静态路网中求解最短路径最有效的直接搜索方法，也是许多其他问题的常用启发式算法。注意——是最有效的直接搜索算法，之后涌现了很多预处理算法（如 CH），在线查询效率是 A*算法的数千甚至上万倍。

- 公式表示为： $f^*(n)=g^*(n)+h^*(n)$ ：

其中， $f^*(n)$ 是从初始状态经由状态 n 到目标状态的最小代价估计，

$g^*(n)$ 是在状态空间中从初始状态到状态 n 的最小代价，

$h^*(n)$ 是从状态 n 到目标状态的路径的最小估计代价。

（对于路径搜索问题，状态就是图中的节点，代价就是距离）

- 真实 $h(n)$ 的选取：

保证找到最短路径（最优解的）条件，关键在于估价函数 $f(n)$ 的选取（或者说 $h(n)$ 的选取）。

- 以 $h(n)$ 表达状态 n 到目标状态估计的距离，那么 $h(n)$ 的选取大致有如下三种情况：

如果 $h(n)< h^*(n)$ ，这种情况下，搜索的点数多，搜索范围大，效率低。但能得到最优解。

如果 $h(n)=h^*(n)$ ，此时的搜索效率是最高的。

如果 $h(n)>h^*(n)$ ，搜索的点数少，搜索范围小，效率高，但不能保证得到最优解。

- A*算法的关键点在于 $f(n)$ 函数的确立,启发式函数的好坏在一定程度上影响着搜索的效率与结果。

- A*算法与图搜索类似，都是通过 Open 表与 Close 表来进行搜索。

- A*算法每次取的是 opened 表中 $f(n)$ 最小的节点,也就是说需要对 Opened 表进行排序。

- A*算法有其特有的性质,既如果从 $S \rightarrow S_0$ 存在一条路径,那么 A*算法一定可以找到最优解。

（三） 解法思路

- 状态表示

M, N, K : 传教士人数,野人人,船容量

(m, c, b) : 状态空间

m: 左岸的传教士个数 0-M

c: 代表左岸的野人个数 0-C

b: 船的状态,1 在左岸,0 在右岸 0or1

目标状态: (0,0,0)

- 约束条件:

当船上传教士数量>0 时,野人数目<=传教士数目

野人数目+传教士数目<船的容量

野人数目!=传教士数目!=0

- 变换操作:

通过减少和增加传教士或野人的数量来拓展节点。

当船在左岸 (boat=1): ①减少野人 ②减少传教士 ③减少野人和传教士

当船在右岸 (boat=0): ①增加野人 ②增加传教士 ③增加野人和传教士

- 启发式函数的定义

$f(n)=d(n)+w(n)$

$d(n)$ 为搜索树的深度,也可以理解为当前是第几轮循环

$w(n)$ 为当前状态到目标状态的实际最小费用的估计值,

在野人传教士问题中, $d(n)$ 定义为划船次数 step, $w(n)$ 定义为 $m+c-K*boat$

(四) 代码

```
1  import queue
2  import numpy as np
3  import prettytable as pt
4
5  # 数据输入
6  M = int(input("Please enter the number of missionaries:")) # 传教士
7  C = int(input("Please enter the number of savages:")) # 野人
8  K = int(input("Please enter the maximum passenger capacity of the ship:")) # 小船容量
9
10 # 定义状态结点
11 class Node:
12     def __init__(self, m, c, boat, fa, step):
13         self.data = np.array([m, c, boat])
14         self.m = m # m:左岸传教士数量
15         self.c = c # c:左岸野人数量
16         self.boat = boat # 船的状态,1在左岸,0在右岸
17         self.step = step # 划船次数
18         self.func = self.step + m + c - K * boat # f(n)的计算
19         self.fa = fa
```

```

21 # 定义排序函数
22 def sort():
23     temp = opened.queue.copy() # 复制open表用于排序
24     l = len(temp) # open表长度
25     # 按照f函数值从小到大排序,相同时按照step的大小排序(冒泡排序)
26     for i in range(l):
27         for j in range(l):
28             if temp[i].func < temp[j].func:
29                 tmp = temp[i]
30                 temp[i] = temp[j]
31                 temp[j] = tmp
32             if temp[i].func == temp[j].func:
33                 if temp[i].step > temp[j].step:
34                     tmp = temp[i]
35                     temp[i] = temp[j]
36                     temp[j] = tmp
37     opened.queue = temp
38
39
40 opened = queue.Queue() # open表
41 snode = Node(M, C, 1, None, 0) # 初始状态节点
42 end_state = np.array([0, 0, 0]) # 目标状态
43 opened.put(snode) # 将初始结点加入open表
44 closed = {} # 初始时closed表为空
45
46
47 # 搜索,类似dfs的写法
48 def search(xnode):
49     temp = opened.queue.copy() # 复制open表
50     for i in range(len(temp)):
51         data = temp[i]
52         now_data = xnode.data
53         if (data == now_data).all():
54             data_func = temp[i].func
55             now_data_func = xnode.func
56             if data_func <= now_data_func:
57                 return False
58             else:
59                 print('')
60                 temp[i] = xnode
61                 opened.queue = temp # 回溯
62                 return True
63     temp.append(xnode)
64     opened.queue = temp # 回溯
65     return True
66
67
68 def Toint(num): # 高精度
69     val = 0
70     for i in num:
71         val = val * 10 + i
72     return val
73
74
75 def Issafe(node): # 判断两岸是否安全
76     '判断当前的节点 两岸是否安全'
77     m, c, boat = node.data
78     if m < c and m != 0: # 若传教士不为0则左岸的野人人数不得超过传教士
79         return False
80     if M - m < C - c and m != M: # 若传教士不为0右岸的野人人数不得超过传教士
81         return False
82     if m < 0 or c < 0: # 人数不得小于0
83         return False
84     if m >= c or M - m >= C - c:
85         return True
86
87

```

```

88 # 划船
89 def method_a_func():
90     while len(opened.queue) != 0:
91         node = opened.get()
92         if (node.data == end_state).all():
93             return node
94         closed[Toint(node.data)] = 1 # 取出的点加入closed表中
95         for i in range(M + 1): # 传教士上船
96             for j in range(C + 1): # 野人上船
97                 # 船上非法情况
98                 if i + j == 0 or i + j > K or (i != 0 and i < j):
99                     # 船运载为空、船超载、船上传教士不为0且野人多于传教士
100                     continue
101                 if node.boat == 1: # 当船在左岸, 下一状态船在右岸
102                     child_node = Node(node.m - i, node.c - j,
103                                         0, node, node.step + 1)
104                 else: # 当船在右岸, 下一状态船在左岸
105                     child_node = Node(node.m + i, node.c + j,
106                                         1, node, node.step + 1)
107                 if Toint(child_node.data) not in closed:
108                     # 如果当前状态不与closed表中的重复
109                     if Issafe(child_node.data):
110                         search(child_node)
111             sort() # 排序
112     return None
113
114
115 result_node = method_a_func() # 获取最终状态路径
116
117
118 #-----输出-----#
119 def output_result(node):
120     all_node = [node]
121     for i in range(node.step):
122         fa_node = node.fa
123         all_node.append(fa_node)
124         node = fa_node
125     return list(reversed(all_node))
126
127
128 def Torow(node, option): # 获取数据行
129     data = node.data
130     m, c, boat = data
131     return [node.step, m, c, option[node.step], M - m, C - c]
132
133
134 def out_option(node_list):
135     option = []
136     for i in range(len(node_list) - 1):
137         now_m, now_c, now_b = node_list[i].data
138         next_m, next_c, next_b = node_list[i + 1].data
139         mr = abs(next_m - now_m)
140         cr = abs(next_c - now_c)
141         mr = f'{mr} Missionary' if mr else ''
142         cr = f'{cr} Cannibals' if cr else ''
143         status = 'left->right,' + mr + ' ' + \
144                 | cr if now_b == 1 else 'right->left,' + mr + ' ' + cr
145         option.append(status)
146     option.append('Success!')
147     return option
148
149
150 def out_put():
151     node_list = output_result(method_a_func())
152     option = out_option(node_list)
153     tb = pt.PrettyTable() # 美观的输出方式
154     # 当前步数, 左侧传教士, 左侧野人, 走法, 右侧传教士, 右侧野人
155     names = ['step', 'l:m', 'l:c', 'option', 'r:m', 'r:c']
156     tb.names = names
157     for node in node_list:
158         tb.add_row(Torow(node, option))
159     print(tb)
160
161
162 out_put()

```

(五) 运行结果

```
+-----+----> d:; cd 'd:\myy\ai'; & 'C:\Users\myy\AppData\Local\Microsoft\WindowsApps\python3.10
ms-python.python-2022.20.1\pythonFiles\lib\python\debugpy\adapter/../../debugpy\launcher' '54473'
Please enter the number of missionaries:3
Please enter the number of savages:3
Please enter the maximum passenger capacity of the ship:2
+-----+-----+-----+-----+-----+-----+
| Field 1 | Field 2 | Field 3 | Field 4 | Field 5 | Field 6 |
+-----+-----+-----+-----+-----+-----+
| 0 | 3 | 3 | left->right, 2 Cannibals | 0 | 0 |
| 1 | 3 | 1 | right->left, 1 Cannibals | 0 | 2 |
| 2 | 3 | 2 | left->right, 2 Cannibals | 0 | 1 |
| 3 | 3 | 0 | right->left, 1 Cannibals | 0 | 3 |
| 4 | 3 | 1 | left->right,2 Missionary | 0 | 2 |
| 5 | 1 | 1 | right->left,1 Missionary 1 Cannibals | 2 | 2 |
| 6 | 2 | 2 | left->right,2 Missionary | 1 | 1 |
| 7 | 0 | 2 | right->left, 1 Cannibals | 3 | 1 |
| 8 | 0 | 3 | left->right, 2 Cannibals | 3 | 0 |
| 9 | 0 | 1 | right->left, 1 Cannibals | 3 | 2 |
| 10 | 0 | 2 | left->right, 2 Cannibals | 3 | 1 |
| 11 | 0 | 0 | Success! | 3 | 3 |
+-----+-----+-----+-----+-----+-----+
```

```
PS D:\myy\ai> d:; cd 'd:\myy\ai'; & 'C:\Users\myy\AppData\Local\Microsoft\WindowsApps\python
ms-python.python-2022.20.1\pythonFiles\lib\python\debugpy\adapter/../../debugpy\launcher' '54
Please enter the number of missionaries:5
Please enter the number of savages:5
Please enter the maximum passenger capacity of the ship:3
+-----+-----+-----+-----+-----+-----+
| Field 1 | Field 2 | Field 3 | Field 4 | Field 5 | Field 6 |
+-----+-----+-----+-----+-----+-----+
| 0 | 5 | 5 | left->right, 3 Cannibals | 0 | 0 |
| 1 | 5 | 2 | right->left, 1 Cannibals | 0 | 3 |
| 2 | 5 | 3 | left->right, 3 Cannibals | 0 | 2 |
| 3 | 5 | 0 | right->left, 2 Cannibals | 0 | 5 |
| 4 | 5 | 2 | left->right,3 Missionary | 0 | 3 |
| 5 | 2 | 2 | right->left,1 Missionary 1 Cannibals | 3 | 3 |
| 6 | 3 | 3 | left->right,3 Missionary | 2 | 2 |
| 7 | 0 | 3 | right->left, 1 Cannibals | 5 | 2 |
| 8 | 0 | 4 | left->right, 3 Cannibals | 5 | 1 |
| 9 | 0 | 1 | right->left,1 Missionary | 5 | 4 |
| 10 | 1 | 1 | left->right,1 Missionary 1 Cannibals | 4 | 4 |
| 11 | 0 | 0 | Success! | 5 | 5 |
+-----+-----+-----+-----+-----+-----+
```

```
PS D:\myy\ai> d:; cd 'd:\myy\ai'; & 'C:\Users\myy\AppData\Local\Microsoft\WindowsApps\py
ms-python.python-2022.20.1\pythonFiles\lib\python\debugpy\adapter/../../debugpy\launcher
Please enter the number of missionaries:5
Please enter the number of savages:4
Please enter the maximum passenger capacity of the ship:3
+-----+-----+-----+-----+-----+-----+
| Field 1 | Field 2 | Field 3 | Field 4 | Field 5 | Field 6 |
+-----+-----+-----+-----+-----+-----+
| 0 | 5 | 4 | left->right, 3 Cannibals | 0 | 0 |
| 1 | 5 | 1 | right->left, 1 Cannibals | 0 | 3 |
| 2 | 5 | 2 | left->right,3 Missionary | 0 | 2 |
| 3 | 2 | 2 | right->left,1 Missionary | 3 | 2 |
| 4 | 3 | 2 | left->right,3 Missionary | 2 | 2 |
| 5 | 0 | 2 | right->left, 1 Cannibals | 5 | 2 |
| 6 | 0 | 3 | left->right, 3 Cannibals | 5 | 1 |
| 7 | 0 | 0 | Success! | 5 | 4 |
+-----+-----+-----+-----+-----+-----+
```

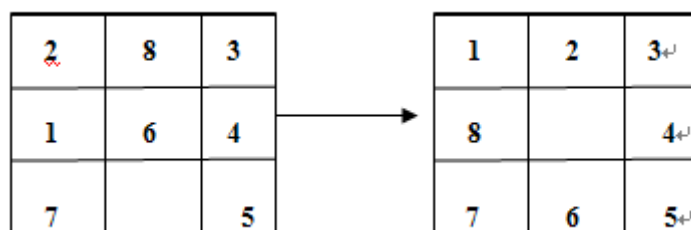
(六) 总结

我在小学时特别沉迷一款网页游戏《奥比岛》，里面有许许多多的益智小游戏，其中一个游戏场景便是与此问题可以说是一模一样的问题，只不过故事的主人公是小奥比与坏人渡河。这是一道非常经典的问题，但如果使用盲目搜索算法，效率非常低下。而 A* 算法优点在于对环境反应迅速，搜索路径直接，是一种直接的搜索算法，因此被广泛应用于路径规划问题。但缺点是实时性差，每一节点计算量大、运算时间长，而且随着节点数的增多，算法搜索效率降低，而且 A* 算法并没有完全遍历所有可行解，所得到的结果不一定是最优解。不过，一旦我们决定此种估价函数得到的解一定是最优解（类似于贪心的证明），那么这种方法便是很好的一种方法。

二、A*算法求解八数码问题

（一） 问题描述

八数码问题也叫九宫问题，是人工智能中状态搜索中的经典问题，其中，该问题描述为：在 3×3 的棋盘，摆有八个棋子，每个棋子上标有 1 至 8 的某一数字，不同棋子上标的数字不相同。棋盘上还有一个空格，与空格相邻的棋子可以移到空格中。要求解决的问题是：给出一个初始状态和一个目标状态，找出一种从初始转变成目标状态的移动棋子步数最少的移动步骤。



（二） 算法介绍

A*（A-Star）算法是一种静态路网中求解最短路径最有效的直接搜索方法，也是许多其他问题的常用启发式算法。注意——是最有效的直接搜索算法，之后涌现了很多预处理算法（如 CH），在线查询效率是 A* 算法的数千甚至上万倍。

- 公式表示为： $f^*(n)=g^*(n)+h^*(n)$ ：

其中， $f^*(n)$ 是从初始状态经由状态 n 到目标状态的最小代价估计，

$g^*(n)$ 是在状态空间中从初始状态到状态 n 的最小代价，

$h^*(n)$ 是从状态 n 到目标状态的路径的最小估计代价。

（对于路径搜索问题，状态就是图中的节点，代价就是距离）

- 真实 $h(n)$ 的选取：

保证找到最短路径（最优解的）条件，关键在于估价函数 $f(n)$ 的选取（或者说 $h(n)$ 的选取）。

- 以 $h(n)$ 表达状态 n 到目标状态估计的距离，那么 $h(n)$ 的选取大致有如下三种情况：

如果 $h(n) < h^*(n)$ ，这种情况下，搜索的点数多，搜索范围大，效率低。但不能得到最优解。

如果 $h(n) = h^*(n)$ ，此时的搜索效率是最高的。

如果 $h(n) > h^*(n)$ ，搜索的点数少，搜索范围小，效率高，但不能保证得到最优解。

• A* 算法的关键点在于 $f(n)$ 函数的确立，启发式函数的好坏在一定程度上影响着搜索的效率与结果。

- A* 算法与图搜索类似，都是通过 Open 表与 Close 表来进行搜索。

- A*算法每次取的是 opened 表中 $f(n)$ 最小的节点,也就是说需要对 Opened 表进行排序。

- A*算法有其特有的性质,既如果从 $S \rightarrow S_0$ 存在一条路径,那么 A*算法一定可以找到最优解。

(三) 解法思路

- 状态表示;

用 3×3 的矩阵表示九宫格, 0 表示空格

- 变换规则;

0 上下左右移动并与相应位置上的数字交换位置

- 搜索方式:

使用启发式搜索, 该问题的评估函数 $f(n) = g(n) + h(n)$ 中, $g(n)$ 可以选用从起始态变换到当前状态移动的次數, $h(n)$ 可以用当前状态距离目标态位置不同的数字 (或空格) 个数表示。

(四) 代码

```
1  import numpy as np
2  import operator
3
4  # 定义状态结点类
5  class State:
6      def __init__(self, m):
7          self.node = m # 节点代表的状态
8          self.f = 0 #  $f(n) = g(n) + h(n)$ 
9          self.g = 0 #  $g(n)$ 
10         self.h = 0 #  $h(n)$ 
11         self.fa = None # 节点的父亲节点
12
13 # 输入
14 n = int(input(("Please enter the number of rows/columns of the square array:")))
15 A = list(input("Please enter the status of the initial matrix:"))
16 B = list(input("Please enter the status of the target matrix:"))
17
18 idx = 0 # 数组下标
19 M = np.zeros((n, n))
20 N = np.zeros((n, n))
21 for i in range(n):
22     for j in range(n):
23         M[i][j] = A[idx]
24         N[i][j] = B[idx]
25         idx = idx + 1
26 opened = [] # open表
27 start = State(M) # 初始状态
28 target = State(N) # 目标状态
29
```

```

31 # 启发函数
32 def h(s):
33     correct = 0
34     for i in range(len(s.node)):
35         for j in range(len(s.node[i])):
36             if s.node[i][j] != target.node[i][j] and s.node[i][j] != 0:
37                 correct += 1
38     return correct
39
40
41 # 对状态节点列表按照估价函数的值排序
42 def sort(l):
43     cmp = operator.attrgetter('f')
44     l.sort(key=cmp)
45
46
47 # A*算法,类似dfs的写法
48 def A_star(xnode):
49     opend = [xnode]
50     falist = [xnode.node] # 父节点
51     while(opend):
52         temp = opend[0]
53         if (temp.node == target.node).all(): # 判断是否与目标节点一致
54             return temp
55         opend.remove(temp) # 将temp移出open表
56         # 判断此时状态的空格位置
57         for a in range(len(temp.node)):
58             for b in range(len(temp.node[a])):
59                 if temp.node[a][b] == 0:
60                     break
61             if temp.node[a][b] == 0:
62                 break
63         # 开始移动
64         for i in range(len(temp.node)):
65             for j in range(len(temp.node[i])):
66                 c = temp.node.copy()
67                 if (i-a)**2 + (j-b)**2 == 1:
68                     c[a][b] = c[i][j]
69                     c[i][j] = 0
70                     new = State(c)
71                     K = 0
72                     for k in range(len(falist)):
73                         if (new.node == falist[k]).all():
74                             break
75                     else:
76                         K = K + 1
77                     if K == len(falist):
78                         new.fa = temp # 更新父节点
79                         new.g = temp.g + 1 # 新节点与父亲节点的距离
80                         new.h = h(new) # 启发函数值
81                         new.f = new.g + new.h # 估价函数值
82                         opend.append(new)
83                         falist.append(new.node)
84
85
86 # 打印
87 def output(f):
88     if f:
89         output(f.fa)
90         print(f.node)
91         print()
92
93 if __name__ == '__main__':
94     flag = A_star(start) #判断是否有解的标记
95     if flag:
96         print("The solution to this problem is:")
97         output(flag)
98     else:
99         print("Sorry! There is no solution to this problem.")

```

（五）运行结果

```
PS D:\myy\ai> d:; cd 'd:\myy\ai'; & 'C:\Users\myy\AppData\Local\Microsoft\WindowsApps\python3.10.exe' 'c:\Users\myy\.vscode\extensions\ms-python.python-2022.20.1\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher' '54838' '--' 'd:\myy\ai\exe2.py'
Please enter the number of rows/columns of the square array:3
Please enter the status of the initial matrix:283164705
Please enter the status of the target matrix:123804765
The solution to this problem is:
[[2. 8. 3.]
 [1. 6. 4.]
 [7. 0. 5.]]

[[2. 8. 3.]
 [1. 0. 4.]
 [7. 6. 5.]]

[[2. 0. 3.]
 [1. 8. 4.]
 [7. 6. 5.]]

[[0. 2. 3.]
 [1. 8. 4.]
 [7. 6. 5.]]

[[1. 2. 3.]
 [0. 8. 4.]
 [7. 6. 5.]]

[[1. 2. 3.]
 [8. 0. 4.]
 [7. 6. 5.]]

PS D:\myy\ai> █
```

```
PS D:\myy\ai> d:; cd 'd:\myy\ai'; & 'C:\Users\myy\AppData\Local\Microsoft\WindowsApps\python3.10.exe' 'c:\Users\myy\.vscode\extensions\ms-python.python-2022.20.1\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher' '54881' '--' 'd:\myy\ai\exe2.py'
Please enter the number of rows/columns of the square array:3
Please enter the status of the initial matrix:283104765
Please enter the status of the target matrix:123804765
The solution to this problem is:
[[2. 8. 3.]
 [1. 0. 4.]
 [7. 6. 5.]]

[[2. 0. 3.]
 [1. 8. 4.]
 [7. 6. 5.]]

[[0. 2. 3.]
 [1. 8. 4.]
 [7. 6. 5.]]

[[1. 2. 3.]
 [0. 8. 4.]
 [7. 6. 5.]]

[[1. 2. 3.]
 [8. 0. 4.]
 [7. 6. 5.]]
```

（六）总结

与实验一同理，本问题使用 A*算法解决也是很好的。A*算法的关键就在于

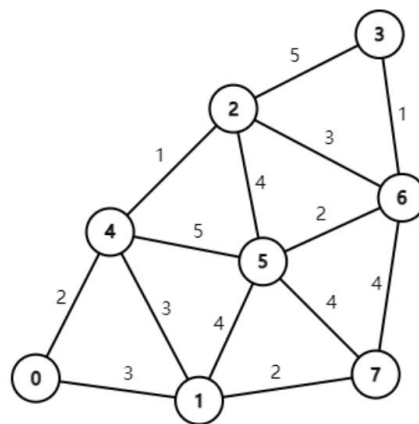
如何寻找合适的估价函数。经过这两次实验，我对使用 open、close 表记录路径更为熟练。

三、自选题目：模拟退火算法求解 TSP

（一）问题描述

旅行商问题，即 TSP 问题（Traveling Salesman Problem）又译为旅行推销员问题、货郎担问题。假设有一个旅行商人要拜访 n 个城市，他必须选择所要走的路径，路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市。路径的选择目标是要求得的路径路程为所有路径之中的最小值。

以下图为例，求从 0 号城市出发访问每一座城市并回到 0 号城市的最短回路：



（二）算法介绍

模拟退火算法来源于固体退火原理，将固体加温至充分高，再让其徐徐冷却，加温时，固体内部粒子随温升变为无序状，内能增大，而徐徐冷却时粒子渐趋有序，在每个温度都达到平衡态，最后在常温时达到基态，内能减为最小。根据 Metropolis 准则，粒子在温度 T 时趋于平衡的概率为 $e(-\Delta E/(kT))$ ，其中 E 为温度 T 时的内能， ΔE 为其改变量， k 为 Boltzmann 常数。用固体退火模拟组合优化问题，将内能 E 模拟为目标函数值 f ，温度 T 演化成控制参数 t ，即得到解组合优化问题的模拟退火算法：由初始解 i 和控制参数初值 t 开始，对当前解重复“产生新解→计算目标函数差→接受或舍弃”的迭代，并逐步衰减 t 值，算法终止时的当前解即为所得近似最优解，这是基于蒙特卡罗迭代求解法的一种启发式随机搜索过程。退火过程由冷却进度表(Cooling Schedule)控制，包括控制参数的初值 t 及其衰减因子 Δt 、每个 t 值时的迭代次数 L 和停止条件 S 。

• 模拟退火算法的模型

1 模拟退火算法可以分解为解空间、目标函数和初始解三部分。

2 模拟退火的基本思想：

(1) 初始化：初始温度 T (充分大)，初始解状态 S (是算法迭代的起点)，每个 T 值的迭代次数 L

- (2) 对 $k=1, \dots, L$ 做第(3)至第 6 步:
- (3) 产生新解 S'
- (4) 计算增量 $\Delta T=C(S')-C(S)$, 其中 $C(S)$ 为评价函数
- (5) 若 $\Delta T < 0$ 则接受 S' 作为新的当前解, 否则以概率 $\exp(-\Delta T/T)$ 接受 S' 作为新的当前解.
- (6) 如果满足终止条件则输出当前解作为最优解, 结束程序。
终止条件通常取为连续若干个新解都没有被接受时终止算法。
- (7) T 逐渐减少, 且 $T \rightarrow 0$, 然后转第 2 步。

- 模拟退火算法的步骤

模拟退火算法新解的产生和接受可分为如下四个步骤:

第一步是由一个产生函数从当前解产生一个位于解空间的新解; 为便于后续的计算和接受, 减少算法耗时, 通常选择由当前新解经过简单地变换即可产生新解的方法, 如对构成新解的全部或部分元素进行置换、互换等, 注意到产生新解的变换方法决定了当前新解的邻域结构, 因而对冷却进度表的选取有一定的影响。

第二步是计算与新解所对应的目标函数差。因为目标函数差仅由变换部分产生, 所以目标函数差的计算最好按增量计算。事实表明, 对大多数应用而言, 这是计算目标函数差的最快方法。

第三步是判断新解是否被接受, 判断的依据是一个接受准则, 最常用的接受准则是 Metropolis 准则: 若 $\Delta T < 0$ 则接受 S' 作为新的当前解 S , 否则以概率 $\exp(-\Delta T/T)$ 接受 S' 作为新的当前解 S 。

第四步是当新解被确定接受时, 用新解代替当前解, 这只需将当前解中对应于产生新解时的变换部分予以实现, 同时修正目标函数值即可。此时, 当前解实现了一次迭代。可在此基础上开始下一轮试验。而当新解被判定为舍弃时, 则在原当前解的基础上继续下一轮试验。

模拟退火算法与初始值无关, 算法求得的解与初始解状态 S (是算法迭代的起点) 无关; 模拟退火算法具有渐近收敛性, 已在理论上被证明是一种以概率 1 收敛于全局最优解的全局优化算法; 模拟退火算法具有并行性。

(三) 解法思路

① 解的编码

求解 TSP 模型采用自然数编码, 0 节点代表初始城市, 1-n 节点表示其他 n 座城市。该问题的解应为 1-n 的一

② 确定初始解

随机生成 1-n 的排列。

③ 邻域新解的生成

采用两点逆转的方法, 即对数列设置两个点, 两点之间的数顺序翻转改变数列。

④ 确定能量函数

以路径长度为能量函数。

⑤ 新解接受准则

采用标准的 Metropolis 准则。

$$P = \begin{cases} 1, E(n+1) < E(n) \\ e^{-\frac{E(n+1)-E(n)}{T}}, E(n+1) \geq E(n) \end{cases}$$

若 $\Delta E < 0$ ，接受新解作为当前解；否则，按照概率判断是否接受新解。

⑥ 搜索最优解

在所有生成的可行解中搜索能量最低的解。

（四） 代码

```
1  import numpy as np
2  import math
3  import random
4  import networkx as nx
5
6
7  # 生成初始解
8  def initial(num): # num为其他城市数量
9      init_path = np.random.choice(
10         list(range(1, num + 1)), replace=False, size=num)
11      init_path = init_path.tolist()
12      return init_path
13
14  # 邻域新解的生成
15  def gene_point(path):
16      num = len(path)
17      # 随机生成两个不重复的点
18      lrpos = np.random.choice(list(range(num)), replace=False, size=2)
19      lpos = min(lrpos[0], lrpos[1]) # 以小的为翻转左界
20      rpos = max(lrpos[0], lrpos[1]) # 以大的为翻转右界
21      # 两点逆转
22      mid_reversed = path[lpos:rpos + 1]
23      mid_reversed.reverse()
24      # 合并得到新路径
25      new_path = path[:lpos]
26      new_path.extend(mid_reversed)
27      new_path.extend(path[rpos + 1:])
28      return new_path
29
30
31  # 能量函数
32  def get_e(path, floyd_matrix): #
33      num = len(path) - 1
34      length = floyd_matrix[0][path[0]] + floyd_matrix[path[-1]][0]
35      for i in range(num):
36         length += floyd_matrix[path[i]][path[i + 1]]
37      return length # 以路径长度为能量函数
```

```

40 # 标准的Metropolis准则
41 def metropolis(e, new_e, t):
42     if new_e <= e:
43         return True
44     else:
45         p = math.exp((e - new_e) / t)
46         return True if random.random() < p else False
47
48
49 # 在所有可行路径中搜索最优解
50 def search(all_path, floyd_matrix): # floyd_matrix可以采用floyd求最短路得到该矩阵
51     min_e = 0xffff # 初始化
52     best_path = all_path[0]
53     for path in all_path:
54         e = get_e(path, floyd_matrix)
55         if e < min_e:
56             min_e = e
57             best_path = path
58     return best_path
59
60
61 #模拟退火算法函数
62 def sa(t0, t1, alpha, inner_iter, num, floyd_matrix):
63     # t0初始温度, t1终止温度, alpha冷却系数, inner_iter内层迭代次数
64     all_path = [] # 初始化解空间
65     all_e = [] # 初始化解的能量函数
66     init = initial(num) # 得到初始解
67     all_path.append(init) # 添加初始解
68     all_e.append(get_e(init, floyd_matrix)) # 添加初始函数
69     t = t0 # 设置初始温度
70     while t > t1: # 循环退火
71         path = search(all_path, floyd_matrix) # 寻找最优解
72         e = get_e(path, floyd_matrix) # 更新能量函数
73         for i in range(inner_iter): # 迭代
74             new_path = gene_point(path) # 产生新路径
75             new_e = get_e(new_path, floyd_matrix) # 计算新能量函数
76             if metropolis(e, new_e, t): #采用标准准则判断
77                 path = new_path
78                 e = new_e
79         all_path.append(path) # 添加新的解
80         all_e.append(e) # 添加新的能量函数
81         t = alpha * t # 更新温度
82     return all_path, all_e # 返回解空间与能量函数
83
84 def floyd(): # 经典floyd算法
85     graph = nx.Graph()
86     graph.add_nodes_from(range(0, 8))
87     edges = [(0, 1, 3), (0, 4, 2), (1, 4, 3),
88             (1, 5, 4), (1, 7, 2), (2, 3, 5),
89             (2, 4, 1), (2, 5, 4), (2, 6, 3),
90             (3, 6, 1), (4, 5, 5), (5, 6, 2),
91             (5, 7, 4), (6, 7, 4)] # (u,v,w)表示u与v城市间有一条权值为w的路径
92     graph.add_weighted_edges_from(edges)
93     min_len = dict(nx.shortest_path_length(graph, weight = 'weight'))
94     floyd_matrix = []
95     for i in range(0, 8):
96         ij = []
97         for j in range(0, 8):
98             ij.append(min_len[i][j])
99         floyd_matrix.append(ij)
100    return floyd_matrix

```

```
102 if __name__ == '__main__':
103     floyd_matrix = floyd()
104     all_path, all_e = sa(3000, pow(10, -1), 0.98, 200, 7, floyd_matrix)
105     print('The solution of the TSP problem is that the cities that the traveling merchants pass by in turn are 0->',
106           search(all_path, floyd_matrix), '->0\nThe best path length is', round(get_e(search(all_path, floyd_matrix), floyd_matrix)))
```

（五）运行结果

```
PS D:\myy\ai> d:; cd 'd:\myy\ai'; & 'C:\Users\myy\AppData\Local\Microsoft\WindowsApps\python3.10.exe' 'c:\Users\myy\.vscode\extensions\20.1\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher' '56500' '--' 'd:\myy\ai\exe3.py'
The solution of the TSP problem is that the cities that the traveling merchants pass by in turn are 0-> [1, 7, 5, 3, 6, 2, 4] ->0
The best path length is 19
```

（六）总结

第一次听说模拟退火算法，是在大一 ACM 竞赛训练。对于此算法，大概只是模糊的知道这是一种随机化算法，对于其细节也并不是很了解，甚至觉得有点玄幻：确定性的问题，居然还可以随机化解决？感觉像是不会做题而随便开始蒙答案——找一些自己认为对的答案，然后挑一个最对的。于是，在课堂上再一次接触到这个算法后，带着从前的一些疑问，我决定对该算法进行学习并解决一个较为熟悉但是一直听说没有合适解法的问题——TSP 问题。虽然没有做大量数据测试，但是通过阅读一些文献资料，确确实实感受到了它的魅力所在，希望以后能够在其他地方有所使用。